Foundation Model Project: FemtoLlama

Student: Marion Forrest
Team: 8
Code Repository: https://github.com/TheAdaptoid/FemtoLlama

Abstract

This work details the design and implementation of FemtoLlama, a collection of compact language models inspired by the architecture of the LLaMA series. Four model variants we constructed to explore parameter scaling across dimensions, layers, and attention heads. The high dimension variant achieved the strongest performance among the four.

Introduction

Large language models (LLMs) and transformer architectures have emerged as central innovations in contemporary machine learning research. Their success is attributable not only to the effectiveness of attention mechanisms, but also to a series of incremental optimizations that collectively enhance model performance. These refinements made by researchers over the past few years have played a critical role in advancing the efficiency and scalability of transformer-based systems.

Within this landscape, the first set of LLaMA models represented a notable contribution. Developed by Meta AI, this family of models consolidated several architectural improvements that had been proposed in transformer literature up to that point, integrating them into a cohesive framework that advanced the state of the art [1].

This paper introduces FemtoLlama, a family of compact language models inspired by the architecture used within LLaMA. The primary objective of this project is to identify, explain, and re-implement the key architectural decisions made by Meta AI researchers and to gain deeper insight into how different architectural choices influence model performance. In contrast to projects such as TinyLlama, FemtoLlama is not intended to serve as a fully functional chat or text completion system [2]. Rather, the focus of this work is on model architecture and training methodology, without consideration of downstream tasks such as reasoning, question answering, or text generation benchmarks.

The name FemtoLlama was selected to emphasize the deliberately small scale of the model. The prefix Femto- was chosen in part because the other diminutive prefixes Tiny-, Micro-, Nano-, and Pico- had already been adopted in prior projects [2, 3, 4, 5].

LLaMA's Contributions

The original LLaMA paper introduced several notable contributions to the field of large language models. First, it demonstrated that state-of-the-art performance could be achieved using only publicly available datasets, totaling approximately 1.4 trillion tokens, thereby eliminating reliance on proprietary corpora [1]. This openness was coupled with a design philosophy that emphasized efficiency. Smaller models such as LLaMA-13B were shown to outperform much larger baselines like GPT-3 (175B), while LLaMA-65B achieved competitive results against models including Chinchilla-70B and PaLM-540B [1].

In addition to dataset and scaling innovations, the LLaMA models incorporated a range of architectural and training optimizations. Techniques such as pre-normalization, SwiGLU activation functions, rotary positional embeddings (RoPE), and optimized attention mechanisms collectively enabled stable training, improved computational efficiency, and faster inference [1]. The models also achieved strong performance across diverse tasks, including commonsense reasoning, closed-book question answering, reading comprehension, mathematical reasoning, and code generation [1].

Pre-Training

Data Sourcing

The training corpus was derived from the August 2018 English Wikipedia dump, restricted to sentences appearing in the introductory sections of content pages [6]. This dataset was chosen for its broad lexical variety and contextual diversity. From this collection of approximately 7.8 million sentences, 5,000 samples were randomly selected. The sample size was constrained by the hardware available for this project.

Tokenization

The raw text was first standardized through a simple preprocessing step in which newline and tab characters were replaced with spaces, yielding a single clean corpus. A Byte-Pair Encoding (BPE) tokenizer was then constructed and trained using the tokenizers library. This choice was made in alignment with the LLaMA models, which also employed BPE as their tokenization

strategy [1]. The tokenizer pipeline included normalization, pre-tokenization by whitespace and punctuation, and digit splitting, whereby all numbers were decomposed into individual digits. The BPE model was initialized with maximum vocabulary of 10,000 and training was performed with a minimum frequency threshold of two.

Following tokenization, the corpus was prepared for causal next-token prediction. The cleaned sentences were concatenated into a continuous token stream, which was then partitioned into training and test splits at a ratio of 9 to 1. Sequences of length 512 were generated using overlapping windows with a stride of 256, ensuring that each example contained 513 tokens (sequence length plus one target token). This procedure yielded a total of 139,058 tokens across the training and test datasets.

Split	Sequences	Tokens
Train	487	125,152
Test	53	13,906

> Table 1: Sequence and token count for the training and validation datasets.

Model Architecture

Four model variants were constructed to allow for comparative analysis across different parameter configurations and to gain some basic insight which direction of scaling improves model performance the most. This design choice was motivated in part by the example set by LLaMA, which was itself released as a family of four models [1]. The parameters of each FemtoLlama variant are summarized in Table 2.

Model	Total Parameters	Heads	Layers	Dimensions
Control	8,276,560	4	4	256
More Heads	8,276,560	8	4	256
More Layers	11,432,864	4	8	256
More Dimensions	22,846,632	4	4	512

> Table 2: Model variants and parameters

All models are based on the standard transformer architecture, with each variant employing multi-head attention and a two-layer feed-forward network within each transformer block. Consistent with the LLaMA design, the feed-forward networks are dimensioned at $\frac{8}{3} \cdot d_{\text{model}}$ [1]. Each FemtoLlama variant incorporates the following four principal architectural optimizations compiled in LLaMA.

Root Mean Square Normalization

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\mathbb{E}[x^2] + \epsilon}}$$

LLaMA used RMSNorm instead of standard LayerNorm. RMSNorm does not perform mean-centering, it instead normalizes activations solely by their root mean square. This simplification reduces computational overhead while preserving training stability. Additionally, by avoiding mean subtraction, RMSNorm is invariant to input mean shifts, which can improve robustness [7].

Pre-Normalization

The Meta AI researchers placed the normalization step before the attention and feed-forward sub-modules, rather than after the addition of the sub-module output and residual connection. This configuration helps stabilize training by mitigating issues such as vanishing or exploding gradients. This adjustment also encourages the model to learn more meaningful representations, as the sub-modules operate on consistently scaled inputs.

At the time of the LLaMA 1 release, pre-norm was not yet widely recognized as superior to the traditional post-norm approach. However, subsequent studies demonstrated that the pre-norm configuration generally yield improved stability and performance in transformer architectures [8].

SwiGLU Activation Function

$$\begin{aligned} \text{SwiGLU}(x) &= \text{Swish}(W_1 x) \cdot W_2 x \\ \text{Swish}(x) &= x \cdot \text{Sigmoid}(x) \end{aligned}$$

The SwiGLU is an activation function combines linear transformations with multiplicative gating to enhance nonlinear expressiveness. The function makes use of two linear projections: one produces values, and the other generates gates that modulate those values through the Swish function.

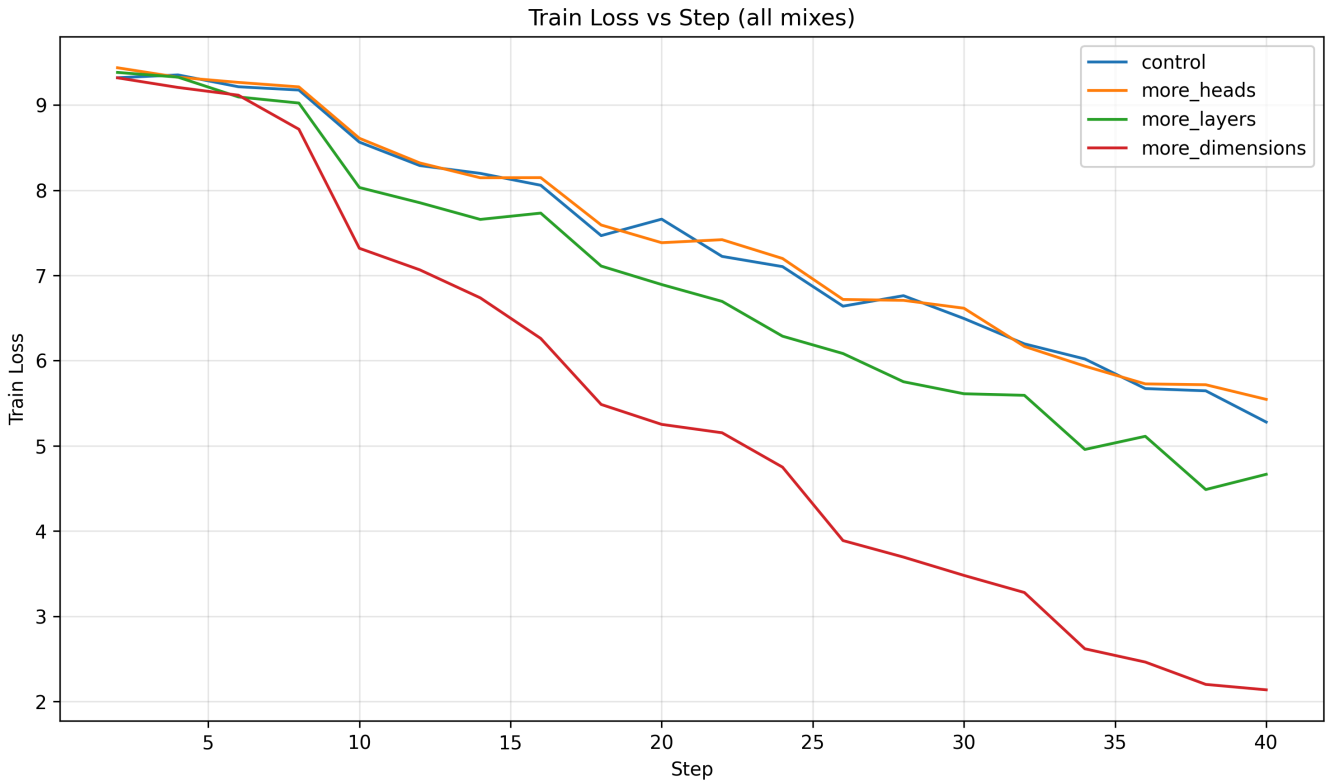
SwiGLU offers smoother gradients and greater representational capacity, compared to traditionally used activation functions like GeLU and ReLU [9]. The multiplicative gating mechanism allows the model to capture more complex interactions, while the use of Swish ensures differentiability and stability [9]. Importantly, these benefits are achieved without a significant increase in computational cost [1].

Rotary Positional Embeddings

Rotary Positional Embeddings (RoPE) encode token positions by rotating query and key vectors in attention space using a position-dependent rotation matrix. Instead of adding positional vectors to embeddings, RoPE applies a complex-number rotation that preserves relative positions naturally. This allows the model to represent the distance between tokens directly within the attention dot product. RoPE also enables improved extrapolation to longer sequence lengths compared to absolute positional encodings which is particularly advantageous for large language models [10].

Training

The models were trained using the AdamW optimizer and a batch size of 64. The Meta AI researchers used a cosine learning rate schedule with gradient clipping, but for ease of implementation, this work used a fixed learning rate of $3e^{-4}$ and did not implement gradient clipping. Each model was trained for 5 epochs with 8 evaluation steps within each epoch.



> Figure 1: Training loss over evaluation step.

Results

Evaluation Metrics

Performance of the FemtoLlama variants were evaluated with the following metrics: next token accuracy, final validation loss, model perplexity, and inference latency. Next token accuracy is a direct measure of the model's ability to capture linguistic patterns. Final validation loss reflects the model's generalization capability by indicating how well it performs on unseen data at the conclusion of training. Perplexity is a commonly used metric in language modeling that measures the uncertainty of the model's predictions. Lower values correspond to more confident and accurate token generation. Inference latency, reported in milliseconds, provided insights into model efficiency and responsiveness.

Model	Next Token Accuracy	Final Validation Loss	Perplexity	Inference Latency (ms)
Control	13.21%	7.3967	1630.66	**25.73**

Model	Next Token Accuracy	Final Validation Loss	Perplexity	Inference Latency (ms)
More Heads	13.21%	7.4816	1775.00	28.31
More Layers	13.21%	7.3691	1586.13	40.19
More Dimensions	13.21%	**7.2060**	**1347.52**	58.85

> Table 3: Model performance comparison table

Analysis

Given the small sizes of the models (less than 100 million parameters) next token accuracy was expected to be low. None of the model variants achieved an accuracy higher than 14%. Nevertheless, validation loss and perplexity still provided meaningful indications of relative performance. Focusing on these two metrics, the high dimension model variant performed the best with the lowest loss and perplexity at 7.2 and 1347.52 respectively. The reduction in perplexity suggests that increasing the model dimension enhances representational capacity and allows the model to better encode semantic relationships within language. However, these gains came at significant computational cost. The extra dimensions increased inference latency by 128% over the control variant.

The model variant with an increased number of attention heads performed worse than the control variant. This made be due to the reduced dimensionality available per attention head, limiting each head’s ability to capture meaningful semantic patterns. The model variant with more transformer layers achieved modest improvements in perplexity relative to the control, though at the expense of increased inference latency which can be attributed to the model's increased depth.

Despite the relative dominance of the high dimension variant, its performance remains far from state of the art. For comparison, the smallest LLaMA model reported training losses below 1.9 and perplexity scores as low as 5.68 [1, 11]. These results showcase both the potential and the limitations of architectural scaling for small transformer systems.

Conclusion

This work details the design and implementation of FemtoLlama, a collection of compact language models inspired by the architecture of the LLaMA series. Four model variants we constructed to explore the effects of parameter scaling across dimensions, layers, and attention heads. The models were evaluated using next token accuracy, validation loss, perplexity, and inference latency. While the high dimension variant achieved the strongest performance among the four variants, these results were still far from the state of the art. Overall, this project provided tremendous insight into how architectural refinements influence performance in transformer-based language models.

Hardware Limitations

This project really brought to light the real compute demands of training language models. Model training was conducted on an above average last-gen desktop computer without any GPU acceleration. At the onset of this endeavor, the lack of GPU compute was recognized as a potential bottleneck, however, the severity of the bottleneck was greatly under estimated. This project had to be scaled back significantly after the desktop ran out of memory several times.

Future Work

The original LLaMA models were released in early 2023. Since then, numerous new architectural optimizations have be introduced. A future iteration of the project could seek to implement additional model variants with mechanisms like Mixture-of-Experts or Mutli-head Latent Attention. A future work could also seek to more closely follow the original training methodology outlined in the LLaMA paper.

References

1. LLaMA: Open and Efficient Foundation Language Models
2. TinyLlama
3. MicroLlama
4. nano-llama
5. PicoLlama
6. Wikipedia Sentences: Collection of 7.8 million sentences from the August 2018 English Wikipedia dump.

7. Root Mean Square Layer Normalization
8. Peri-LN: Revisiting Layer Normalization in the Transformer Architecture
9. GLU Variants Improve Transformer
10. RoFormer: Enhanced Transformer with Rotary Position Embedding
11. A Perplexity Benchmark of llama.cpp